

פתרון בחינה בקורס מבני נתונים ומבוא לאלגוריתמים (76689)
סמסטר א' מועד א' - תשפ"א

רועי כנעני

פתרון שאלה 1

(א)

א. אלגוריתם איטרטיבי הבודק אם עץ בינרי T הוא עץ חיפוש בינרי:

```
CHECK-BST ( $T$ )  
 $x \leftarrow \text{TREE-MINIMUM}(T)$   
 $i \leftarrow 1, bst \leftarrow \text{TRUE}$   
while  $i \leq n - 1$  and  $bst = \text{TRUE}$   
    do  $y \leftarrow \text{TREE-SUCCESSOR}(x)$   
       if  $\text{key}[x] \leq \text{key}[y]$   
           then  $i \leftarrow i + 1$   
               $x \leftarrow y$   
       else  $bst \leftarrow \text{FALSE}$   
return  $bst$ 
```

לפי שאלה 13.2-4 בספר, זמן הריצה של האלגוריתם הוא $\Theta(n)$.

פתרון יפה ואלגנטי נוסף לשאלה 1א' (עומר):

[illegible]

1ב) חישוב חסם עליון וחסם תחתון אסימפטוטיים הדוקים ככל האפשר עבור $T(n)$ הבאה:

$$T(n) = 7T\left(\frac{n}{2}\right) + n^2 \cdot (\lg n)^5 + (\lg n)^8$$

פתרון:

ניתן לפתור בשיטת האב.

$$f(n) = n^2 \cdot (\lg n)^5 + (\lg n)^8 = \theta(n^2 \cdot (\lg n)^5)$$

מתקיים:

$$a = 7, b = 2 \quad \log_b a = 2.8 \quad (< 3)$$

ולכן, לפי שיטת האב מתקיים מקרה א'

$$n^2 \cdot (\log n)^5 = O(n^{\log_2 7 - \varepsilon}) = O(n^{2.8 - \varepsilon})$$

כאשר אפסילון יכול להיות $\varepsilon = 0.1$ (או אפילו $\varepsilon = 0.5$) עבור ח-ים גדולים מספיק.

כלומר האיבר $7T\left(\frac{n}{2}\right)$ הוא האיבר הדומיננטי של נוסחת הנסיגה. נקבל

$$T(n) = \theta(n^{\log_2 7}) = \theta(n^{2.8})$$

פתרון שאלה 2

ראשית, נמצא את המקסימום (האיבר ה- $\frac{n}{2^0}$) במעבר אחד על המערך – $O(n)$. ואז נעבוד רקורסיבית: נמצא את החציון (האיבר ה- $\frac{n}{2^1}$) בזמן $O(n)$ בעזרת quick select הדטרמיסטי הלינארי שלמדנו בכיתה. באותה מסגרת זמן נוכל גם לחלק את המערך לשני חלקים – איברים גדולים מהחציון ואיברים קטנים מהחציון. מכאן נמשיך ברקורסיה עם חצי המערך השמאלי. סיבוכיות זמן: נסמן ב- $c \cdot n$ את החסם האסימפטוטי על זמן הריצה של quick select. האלגוריתם שלנו ירוץ בזמן

$$c \cdot n + c \cdot \frac{n}{2} + c \cdot \frac{n}{4} + \dots + c \cdot \frac{n}{n} = cn \left(1 + \frac{1}{2} + \frac{1}{4} + \dots + \frac{1}{n} \right) \leq 2cn = O(n)$$

טעויות נפוצות:

- הפעלת הרקורסיה על שני חצאי המערך.
- מיון ב- $O(n \log n)$ – לא יעיל.
- שימוש במיון לינארי (counting sort, radix sort) – לא יעיל כאשר אין שום מידע מקדים על הקלט.
- שימוש בעצים בינאריים – בערימות מקס/מיני אין דרך לדעת את מיקומו של החציון. כנ"ל בעץ חיפוש בינארי. עץ AVL היה יכול לעזור אבל בנייתו ממערך לא ממיון תיקח $O(n \log n)$.

פתרון שאלה 3

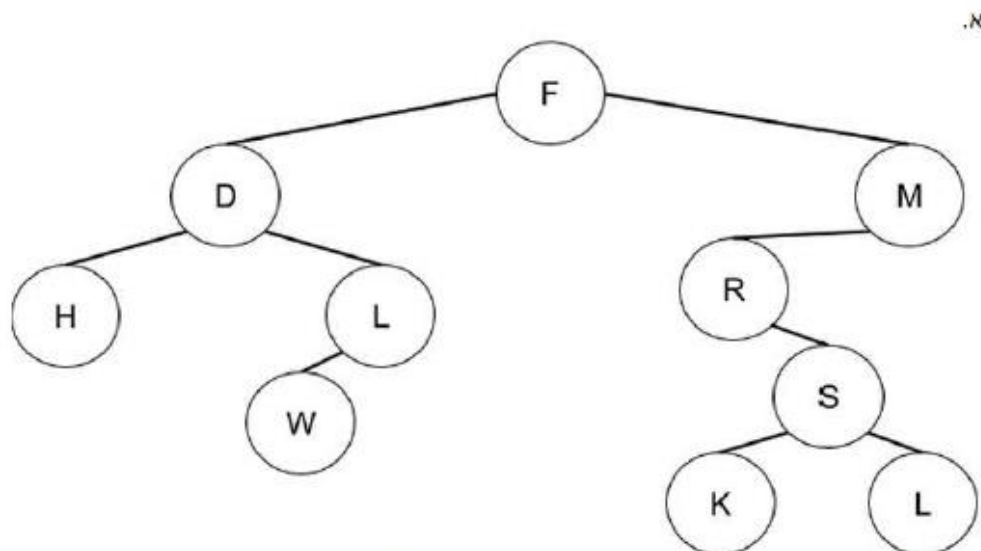
הערה:

(א) פתרון 3א) עבור Preorder של עץ בינארי כללי (משמאל לימין) שכולל את נוספת L (שלא הופיעה בשאלה בבחינה) כלומר:

F D H L W M R S K L

כאשר כל אות מייצגת צומת בעץ. כמו כן, סידור ה Inorder הוא:

H D W L F R K S L M

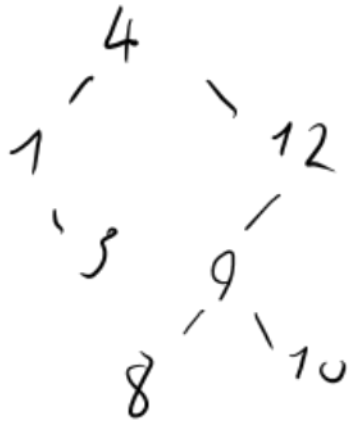


*הלכתי על הסידור בPreorder ובניתי את העץ לפי רמזים מInorder

ב' אין עץ בינארי שונה עם אותם סידורים preorder ו inorder מכיוון שהאלגוריתם הרקורסיבי מחלק בכל שלב את הבעיה לתתי עצים, כלומר F המופיע ראשון בסידור preorder אומר ש F שורש תת העץ ובסידור inorder F מחלק באופן מוחלט את הצמתים בסידור inorder לתת העץ הימני ותת העץ השמאלי לפי כל הצמתים שממין F וכל הצמתים שמשמאל בהתאמה.

ג' התשובה: רק כאשר X הינו עלה. מחיקה של עלה מעץ"ב לא משנה את מבנה העץ וביצוע חיפוש המיקום והכנסת הערך תתבצע לאותו המקום. בכל מקרה אחר ש X לא עלה הכנסתו מחדש תגרום לו להיות עלה.

3-ב) פתרון נוסף (אור)



מתחילים מ-4, שחייב להיות בקודקוד. הולכים קודם כל שמאלה ומדפיסים את הבן השמאלי, והוא אחד. אח"כ הולכים שמאלה, ואם אין כלום הולכים ימינה. מאחר ומדובר בעץ חיפוש בינארי שלוש חייב להיות מימין לאחד.

חוזרים לשורש והולכים ימינה. מדפיסים את הקודקוד (12). 8, 9 ו-10 חייבים להיות משמאל ל-12 בעץ חוקי. הולכים שמאלה, ומאחר והדבר הראשון שעושים זה להדפיס את הצומת ומודפס 9, אז 9 חייב להיות הבן השמאלי של 12. אח"כ מדפיסים 8. מאחר ובכל קריאה רקורסיבית הדבר הראשון שנעשה זה להדפיס את הצומת,

8 חייב להיות בן של 9. הדבר הבא שמדפיסים הוא 10. 10 לא יכול להיות בן של 8 מאחר והוא הוא גדול מהאב הקדמון של 8 (9) ולכן הוא חייב להיות בן ימני של 9, ולעץ יש רק סידור אחד אפשרי.

פתרון שאלה 4

נשתמש בעץ אדום-שחור מורחב (עץ ערכי מיקום) T , כך שבכל צומת x של העץ T נמצאים 2 שדות נוספים: שדה נוסף $size[x]$ שמכיל את מספר הצמתים (הפנימיים) בתת-עץ המושרש בצומת x (כולל הצומת x עצמה). כמו באלגוריתם של עץ ערכי מיקום שנתון בפרק 15 בספר.

ושדה שני נוסף $sum[x]$ שמכיל את סכום מפתחות הצמתים (הפנימיים) בתת-עץ המושרש בצומת x (כולל המפתח של הצומת x עצמה).

ראו תאור בפרק 15 הרחבה של מבנה נתונים בספר הלימוד מבוא לאלגוריתמים (מהדורת 1998, תרגום הספר בהוצאת האו"פ)

or Chap. 14 Augmenting Data Structures

(CLRS) Introduction to Algorithms, 3rd ed 2009.

הערה: (מראה המקום המדויק אינו נדרש בבחינה, אך רצוי לתת את שם הפונקציות המדויק ככל האפשר, כפי שהוא נתון בספר הלימוד).

מימוש הפעולות הנדרשות במבנה הנתונים:

תחזוקת השדה sum בהכנסה והוצאה מעץ אדום-שחור אינה משנה את הסיבוכיות, לפי משפט 14.1 sum (*) הנוסחה לחישוב הערך של sum בצומת x בעץ T היא:

$$sum[x] = sum[left[x]] + sum[right[x]] + key[x]$$

$$sum[nil[T]] = key[nil[T]] = 0$$

תחזוקת השדה sum :

פעולת $INSERT(S, x)$: בכנסת המפתח x לעץ, נוסיף את הערך x לשדה sum בכל צומת במסלול מהשורש ועד לנקודת החיבור של הצומת החדש.

$DELETE(S, z)$: במחיקת האיבר שאליו מצביע z מהמבנה, יהי y הצומת שיוצא בפועל מהעץ:

-נפחית את הערך $key[y]$ מהשדה sum בכל צומת במסלול מ y ועד לשורש.

-אם y שונה מ z , נוסיף את הערך $key[y]-key[z]$ לשדה sum בכל צומת במסלול מ z ועד לשורש.

בשלב התיקונים של הכנסה או הוצאה, בכל פעולת סיבוב (רוטציה) נשתמש בנוסחה (*) הנ"ל לתיקון ערכו של השדה sum בצומת, בדומה לתיקון השדה $size$ בעץ ערכי מיקום.

הפעולה $FIND_k(S, k)$: החזרת המפתח k בגודלו במבנה S – ממומשת ע"י

השגרה $OS-Select(x, i)$ שמחזירה את המפתח ה- i הקטן ביותר בתת עץ ערכי מיקום שמושרש ב- x בזמן של $O(\log n)$

תאור אפשרי של הפעולה Sum_Of_Smallest_k יכול להיות:

נפעיל את הפונקציה

$\text{FIND_k}(S, k)$ להחזרת המצביע x לאיבר ה- k בגודלו.

נאתחל משתנה

$\text{sumx} = x.\text{key} + x.\text{left}.\text{sum}$

כעת נעלה מהצומת x למעלה לכיוון השורש. ובכל פעם

שנעלה מבן ימני לצומת y נוסיף למשתנה הסכום

$\text{sumx} = \text{sumx} + y.\text{key} + y.\text{left}.\text{sum}$

כשנגיע לשורש נחזיר את משתנה הסכום sumx .

פתרון מלא נוסף של סטודנט לשאלה 4 (אור):

מבנה הנתונים שאני מציע הוא עץ אדום שחור מורחב (Augmented Red-Black Tree) שכולל בתוכו את השדה size שראינו בהרצאות ושמופיע גם בדף הנוסחאות. עץ אדום שחור תומך בפעולות $\text{Insert}(S, x)$ ו- $\text{Delete}(S, x)$ בזמן של $O(\log n)$. עץ אדום שחור שמורחב עם size עדיין תומך בפעולות האלו באותו זמן הריצה האסימפטוטי, ובנוסף תומכת בשגרה $\text{OS-Select}(x, i)$ שתחזיר את המפתח ה- i הקטן ביותר בתת עץ שמושך ב- x בזמן של $O(\log n)$. אם נקרא לשגרה הזאת עבור כל העץ היא בעצם תיתן לנו את $\text{Find_k}(S, k)$ עם זמן הריצה האסימפטוטי הרצוי.

כדי להחזיר את $\text{Sum_Of_Smallest}(S, k)$ נרחיב את העץ עם שדה נוסף בכל צומת x שיקרא sum שמכיל את סכום כל האיברים שנמצאים בתת העץ שמושך ב- x ובערך של x עצמו. ההגדרה נראית ככה:

$$x.\text{sum} = x.\text{key} + x.\text{left}.\text{sum} + x.\text{right}.\text{sum}$$

השגרה תעבוד בצורה הבאה:

ראשית, היא תבדוק האם k קטן ממספר האיברים בעץ. אם כן, היא תחזיר את sum של השורש.

אחרת:

- נגדיר משתנה שישמור את הסכום (total_sum).
- המשתנה הזה יקבל ערך ההתחלתי ששווה לערך של x ($x.\text{data}$) ולסכום של תת העץ השמאלי של x ($x.\text{left}.\text{sum}$).
- הפונקציה תחפש את המפתח ה- k הקטן ביותר, ואז ממנו תתחיל לטפס מעלה בלולאה (קריאה מהצורה $x = x.\text{parent}$ בשורה האחרונה של הלולאה) כלפי השורש כאשר בכל צומת היא תבצע את הבדיקה הבאה:
- האם הצומת היא בן ימני של ההורה שלו?
 - כן – נוסיף לסכום (total_sum) את הערך של ההורה ($x.\text{parent}.\text{data}$) ואת סכום תת העץ השמאלי של ההורה ($x.\text{parent}.\text{sum}$).
 - לא – לא נעשה כלום. נמשיך להורה.

הלולאה שתיארתי תעשה שני דברים – קריאה חד פעמית ל- $\text{Find}(S,k)$ שתיקח $O(\log n)$, ואז ריצה בלולאה כלפי מעלה, ולכן זמן הריצה יהיה גובה העץ ($O(h)$), שבעץ אדום שחור נותן זמן ריצה של $O(\log n)$. מאחר והשגרה שהצעתי עושה פעמיים $O(\log n)$, אפשר לחסום גם אותה ע"י $O(\log n)$.

נשאר רק דבר אחד, וזה להוכיח שהשדה שהוספתי לא פוגע בזמן הריצה האסימפטוטי של הפעולות האחרון שפועלות על העץ כגון הכנסה ומחיקה של איברים. בהקשר הזה ספר הלימוד (קורמן) מדגים ומוכיח את זה במשפט (Theorem) 14.1 (עמוד 346 בספר בגרסה האנגלית, מהדורה 3). ע"פ ההוכחה, אפשר להוסיף כל שדה שהתכונה שלו תלויה אך ורק במידע שנמצא בצומת ובשני הבנים של הצומת (ולפעמים גם באותו השדה שמוסיפים בשני הבנים) בלי לפגוע בזמן הריצה האסימפטוטי של פעולות ההכנסה והמחיקה. מאחר והשדה שהצעתי מקיים תנאי זה (תלוי אך ורק במידע שבשדה x ובסכומים של הבנים שלו) זמן הריצה האסימפטוטי צפוי להשמר. ניתן לראות את הדמיון:

$$x.\text{sum} = x.\text{key} + x.\text{left.sum} + x.\text{right.sum}$$

$$x.\text{size} = x.\text{left.size} + x.\text{right.size} + 1$$